locally on port 5000, which is the default port for the Flask app, and you can see the output 'Hello from Kubernetes!' on *http://localhost:5000*.

Once the server is running locally, we will create a Docker image to be used by Kubernetes.

Create a file with the name *Dockerfile* and paste the following code snippet in it:

```
FROM python:3.7

RUN mkdir /app
WORKDIR /app
ADD . /app/
RUN pip install -r requirements.txt

EXPOSE 5000
CMD ["python", "/app/main.py"]
```

The instructions in *Dockerfile* are explained below:

1. Docker will fetch the Python 3.7 image from the Docker hub.
2. It will create an app directory in the image.
3. It will set an app as the working directory.
4. Copy the contents from the app directory in the host to the image app directory.
5. Expose Port 5000.
6. Finally, it will run the command to start the Flask server.

In the next step, we will create the Docker image, using the command given below:

```
docker build -f Dockerfile -t flask-
kubernetes:latest .
```

After creating the Docker image, we can test it by running it locally using the following command:

```
docker run -p 5001:5000 flask-kubernetes
```

Once we are done testing it locally by running a container, we need to deploy this in Kubernetes.

We will first verify that Kubernetes is running using the *kubectl* command. If there are no errors, then it is working. If there are errors, do refer to *https://kubernetes.io/docs/setup/learning-environment/minikube/*.

Next, let's create a deployment file. This is a *yaml* file containing the instruction for Kubernetes about how to create pods and services in a very declarative fashion. Since we have a Flask Web application, we will create a *deployment.yaml* file with both the pods and services declarations inside it.

Create a file named *deployment.yaml* and add the following contents to it, before saving it:

```
apiVersion: v1
kind: Service
metadata:
  name: flask-kubernetes -service
spec:
  selector:
    app: flask-kubernetes
  ports:
  - protocol: "TCP"
    port: 6000
    targetPort: 5000
  type: LoadBalancer


---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-kubernetes
spec:
  replicas: 4
  template:
    metadata:
      labels:
        app: flask-kubernetes
    spec:
      containers:
      - name: flask-kubernetes
        image: flask-kubernetes:latest
        imagePullPolicy: Never
        ports:
        - containerPort: 5000
```

Use *kubectl* to send the *yaml* file to Kubernetes by running the following command:

```
kubectl apply -f deployment.yaml
```

You can see the pods are running if you execute the following command:

```
kubectl get pods
```

Now navigate to *http://localhost:6000*, and you should see the 'Hello from Kubernetes!' message.

That's it! The application is now running in Kubernetes!

What Kubernetes cannot do

Kubernetes is not a traditional, all-inclusive PaaS (Platform as a Service) system. Since Kubernetes operates at the container level rather than at the hardware level, it provides some generally applicable features common to PaaS offerings, such as deployment, scaling, load balancing, logging, and monitoring. Kubernetes provides the building blocks for developer platforms, but preserves user choice and flexibility where it is important.

- Kubernetes does not limit the types of applications supported. If an application can run in a container, it should run great on Kubernetes.
- It does not deploy and build source code.
- It does not dictate logging, monitoring, or alerting solutions.
- It does not provide or mandate a configuration language/system. It provides a declarative API for everyone's use.
- It does not provide or adopt any comprehensive machine configuration, maintenance, management, or self-healing systems. END

Reference

https://www.docker.com/resources/what-container

By: Abhinav Nath Gupta

The author is a software development engineer at Cleo Software India Pvt Ltd, Bengaluru, and is interested in cryptography, data security, cryptocurrency and cloud computing.